

RC Smart Drive 기술서

OpenCV 기반 자율주행 RC카 및 LiDAR 장애물 감지 시스템

1. 프로젝트 개요

개발 목적: 본 프로젝트는 RC카가 지정된 차선(노란색 트랙)을 실시간으로 추적 및 주행하며, 주행 중 전방 장애물을 LiDAR 센서를 활용하여 감지해 즉각적인 비상 정지 및 음성 위험 안내를 수행하는 안전 자율주행 모델을 개발하는 데 목적이 있습니다. 컴퓨터 비전(OpenCV), 센서 처리(LiDAR, 조도센서), 인공지능경망 추론(DNN Lux), 멀티스레딩 아키텍처 및 액추에이터 제어 기술을 단일 프로세스 상에 유기적으로 통합하여 실제 자율주행 차량의 핵심 ADAS 메커니즘을 소형 모빌리티 플랫폼 내에 성공적으로 구현하였습니다.

2. 개발 환경 및 플랫폼

구분	내용
운영체제 (OS)	Ubuntu 22.04 LTS (안정적인 리눅스 커널 기반의 자율주행 환경 제공)
개발 언어	Python (확장성 높은 멀티스레드 구현 및 NumPy 가속 연산 활용)
통합 개발 환경	Visual Studio Code
영상 처리 라이브러리	OpenCV (카메라 입력 프레임 분석 및 영상 차선 트래킹 필터링)
수학 연산	NumPy (배열 가속 연산, 경량 DNN 추론 시 고속 연산에 활용)
오디오 입출력	PyAudio (TTS 음원 비동기 Queue 플레이백 지원)
센서 디바이스	RPLidar, CDS 조도센서
H/W 플랫폼	AutoCar Prime NX (NVIDIA Jetson 기반 소형 자율주행 플랫폼)
동시성 제어	Python Threading Module

3. 시스템 아키텍처 및 하드웨어 구성

본 시스템은 다양한 모듈 간의 독립성 보장 및 무정지 실시간 자율주행을 위해 논블로킹(Non-blocking) 병렬 제어 아키텍처를 가집니다. 각 입출력 디바이스는 물리 계층과 맞물려 실시간 주행 프로세스를 지속합니다.

구성 요소	역할
Camera	프론트 뷰 노란색 차선 인식을 위한 실시간 비디오 프레임 스트리밍 캡처
OpenCV Vision	HSV 변환 및 모멘트 분석을 통한 주행 차선 중심 계산 및 조향 편차값 연산
Main Controller	조향(Steering) 각도와 주행 모터 속도 제어, 동작 상태 관리(Auto/Manual)
LiDAR Sensor	360도 스캔을 기반으로 한 실시간 거리 정보 수집 및 전방 위험 상황 판별
CDS & DNN Engine	조도 센서에서 아날로그 입력 신호를 획득, NumPy 수식 추론을 거쳐 실시간 Lux로 정량화
LED / Audio	Lux 상태에 따른 자동 야간 전조등 발광 및 위급 장애물 감지 TTS 알림

4. 시스템 라이프사이클 및 동작 프로세스

세 가지 핵심 프로세스(차선 트래킹, 긴급 회피, 자동 라이트)가 스레드 풀 환경 내에서 상시 독립 구동하며 차량 조향 및 구동 축을 지배합니다.

[비전 주행 제어 플로우]

Camera -> OpenCV 영상처리 -> 차선 중심 계산 -> PD Steering Controller -> AutoCar Motor 제어

[실시간 충돌 회피 플로우 (동시 실행)]

LiDAR -> 전방 300° ~ 60° 범위 필터링 -> 40cm 이내 장애물 탐지 -> Emergency Stop 구동 -> TTS 비동기 플레이어

[지능형 외부 조도 제어 플로우 (동시 실행)]

CDS Sensor -> 데이터 수집 -> NumPy 순방향 인공신경망 추론(DNN Lux) -> 450 Lux 이하 LED 자동 점등

5. 소프트웨어 동시성 설계 (Multi-threading)

파이썬의 GIL(Global Interpreter Lock) 제약을 고려하면서도, I/O 바운드 작업과 하드웨어 제어 응답 속도를 최대화하기 위해 각 도메인을 독립 스레드로 격리 설계하였습니다.

- **Main Thread:** 시스템의 전반적인 초기화, 스레드 모니터링 및 비상 복구 제어 수행.
- **Main Control Thread:** 차량의 현재 모드(Manual / Auto) 관리 및 최종 구동축/조향 서보 명령 제어.
- **OpenCV Vision Thread:** 카메라 프레임을 초당 30프레임 이상 처리하며 주행 오차값을 PD 제어기에 연속 피딩.
- **LiDAR Thread:** 센서 데이터를 지속 스트리밍하여 감지 각도 이내 위험 영역 모니터링 후 긴급 플래그 세팅.
- **Auto Headlight Thread:** 아날로그 조도 값을 수집, NumPy 뉴럴넷 추론 결과를 통해 LED 상태 제어.
- **Audio Thread:** 충돌 경고 오디오 재생 시 전체 주행 서브 프로세스 지연을 원천 방지하기 위해 Queue 형태로 오디오 처리.

6. 핵심 알고리즘 및 수학적 설계

(1) OpenCV 노란색 차선 추적 알고리즘

수신된 프레임에서 관심 영역(ROI, Region of Interest)을 하단 주행 가능 영역으로 제한하여 불필요한 배경 요소를 배제합니다. 그 후 **Gaussian Blur** 커널 필터를 거쳐 연산 노이즈를 효과적으로 제거하고 차선을 검출합니다.

- **HSV 색공간 필터링**: 조도 변화에 강인하도록 BGR에서 HSV로 변환, 노란색 임계 영역만 필터링하여 이진화 이미지(Binary Mask) 생성.
- **모멘트 기반 중심 계산**: 이진화 처리된 흰색 픽셀 집합의 모멘트(M_{00} M_{10} M_{01})를 계산하여 차선의 공간적 중심 C_x 계산:

$$C_x = M_{10} / M_{00}$$

최종 주행 오차값(Error)은 화면의 가로 중심 기준 $E = C_x - (Width / 2)$ 로 산출됩니다.

(2) PD 제어를 이용한 조향 보정 (PD Steering Control)

단순 비례 제어(Proportional Control) 수행 시, 관성에 의해 조향각이 타겟을 오버슈팅하며 좌우로 크게 흔들리는 헛팅(Hunting) 현상이 발생합니다. 이를 안정화하기 위해 오차 변화율(Derivative)을 반영한 PD 제어를 도출하여 차량의 거동을 감쇄시켰습니다.

$$u(t) = K_p \cdot E(t) + K_d \cdot [E(t) - E(t-1)] / \Delta t$$

여기서 K_p 는 비례 게인, K_d 는 미분 게인이며, 오차의 변화 방향과 크기에 반응하여 급격한 조향 각 변화 시 역방향 댐핑을 가해 안정적으로 중심 트랙을 추종하게 합니다.

(3) 자이로 센서 피드백을 이용한 직진 보정 (ESC Steering Calibration)

서보모터와 바퀴의 물리적 유격 및 무게 중심 불균형 등으로 차량이 기본 주행 상태에서 직진하지 않고 편향되는 문제를 해결하기 위해, 부트 타임 시 측정된 자이로 기준 센서값(**BASE_GYRO**)과 실시간 자이로 센서 오프셋을 비교 계산하여 조향 베이스 오프셋 값을 보정하는 주행 캘리브레이션을 매 회 시행합니다.

(4) LiDAR 기반 전방 각도 세그멘테이션 필터링

360도 전체 센싱 정보를 매 프레임 파싱하는 아키텍처는 리소스 소모가 높습니다. 이를 극복하고자 주행 방향에 상응하는 각도 영역인 $[300^\circ \sim 360^\circ]$ 및 $[0^\circ \sim 60^\circ]$ (즉, 차량 정면 중심 기준 $\pm 60^\circ$ 각도 콘 영역)만 필터 검출 대상으로 지정하였습니다.

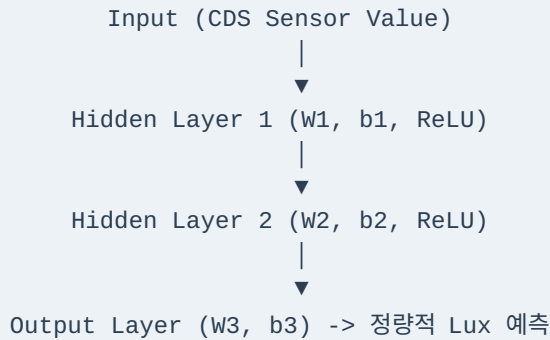
임계 작동 범위: 타겟 검출 각 범위 이내의 반사 거리 필터링 값이 $R \leq 40cm$ 인 경우 즉시 EMERGENCY_STOP_FLAG를 활성화하여 주행 제어 스레드의 출력을 중단시키고 제동 명령을 최우선 전송합니다.

(5) PyAudio 비동기 오디오 큐 아키텍처

알람 재생을 순차 동기식(Synchronous)으로 진행하면 사운드 디스크 I/O 및 재생 시간동안 차선 추적 루프가 락(Lock) 상태에 빠지게 됩니다. 이를 방지하고자, 오디오 이벤트 발생 시 obstacle_alert.wav 파일 데이터를 즉시 전역 오디오 Queue로 인큐(Enqueue)하고, 별도로 루프 구동 중인 오디오 스레드가 해당 큐를 디큐(Dequeue)하여 재생 스트림을 흘리는 논블로킹 패러다임을 확립하였습니다.

(6) 경량 순방향 인공신경망(DNN) 기반 Lux 예측 엔진

TensorFlow나 PyTorch 등 거대한 헤비 프레임워크 로딩을 배제하고, 임베디드 장비의 고속 자원 활용을 위해 미리 훈련된 심층 신경망의 가중치(Weights)와 편향(Biases) 벡터 데이터를 JSON 포맷으로 패키징하여 가져왔습니다. 행렬 연산은 NumPy를 통해 완전히 구체화되었습니다.



예측 결과값 $Lux \leq 450$ 인 어두운 터널이나 구획 진입 시 자동으로 차체 구동 라이트(GPIO LED) 신호를 High로 인에이블(Enable) 하여 자율 가시성을 확보합니다.

7. 개발 중 직면 과제 및 솔루션 (Troubleshooting)

이슈 1: 영상 처리 지연에 따른 주행 조향계 'S'자 거동 (Hunting)

초기 단계에서 연산 과부하에 따른 제어 신호 레이턴시로 인해 차량이 트랙 중심을 이탈 및 요동치는 증상이 식별되었습니다.

- **해결 방안:** 관심 영역(ROI)을 수평 종단부 위주로 추가 극대화 슬라이싱하고 가우시안 블러 커널 사이즈를 경량화 (3x3)하였습니다. 또한 PD 제어 모델을 결합하고 제어 주기 루프 주파수를 보정하여 오실레이션을 수렴시켰습니다.

이슈 2: LiDAR 360° 연산 과부하에 따른 CPU 점유율 증가

전체 회전 각도의 측정 배열을 전수 연산함으로써 메인 Jetson 모듈의 인터럽트 부하 및 CPU 병목 현상이 발생하였습니다.

- **해결 방안:** 주행 방향에만 영향을 미치는 전방 120도 구역만 배열 슬라이싱 단계에서 선택적으로 통과시키는 공간적 마스크 필터를 구현하여 불필요한 원거리 점 데이터 분석 연산을 65% 이상 감소시켰습니다.

8. 주요 성과 및 향후 로드맵

본 RC Smart Drive는 복잡한 임베디드 디바이스 간 독립 프로세스를 파이썬 스레딩 기반으로 성공적으로 정렬하고, 가볍고도 강력한 자율주행 차선 트래킹과 전방 LiDAR 비상 대처 설루션을 완비하는 실증 성과를 냈습니다.

차세대 시스템 개선 방향

- YOLO-Nano 엣지 AI 객체 감지 인공지능 기반 지능형 표지판 인식 및 인체 식별 기능 융합
- Dynamic Path Planning 장애물 인근 통과 우회 회피 조향 알고리즘 수립
- pywebview Dashboard 실시간 시스템 텔레메트리 데이터 웹뷰 UI 관제 연동
- Flask API Server 원격 긴급 모니터링 및 실시간 비디오 피드 백엔드 환경 구축
- Odometry 시각 오도메트리(Visual Odometry)와 센서 융합 기법을 더한 차량 정밀 절대위치 측위(Localization)